PIMPRI CHINCHWAD EDUCATION TRUST's
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING (PCCOE)
Department of Information Technology

ASSIGNMENT NO. 5	**Academic Year:** 2025–2026	**Semester:** II — S.Y. B.Tech
Course: Database Management System Lab	**Course Code:** BIT24PC04	**Division / Batch:** A1
Student Name: Sidhant Mattoo	**PRN:** 124B1F016	**Department:** Information Technology

PL/SQL – CONTROL STRUCTURES & EXCEPTION HANDLING

CASE STUDY: Hospital Management System — PL/SQL Context

Database Schema Overview
All PL/SQL programs in this assignment are built around the Hospital Management System schema established in Assignments 2 and 3. The schema includes the following core tables: Doctor, Patient, Appointment, Bill, and Prescription. Each query demonstrates how anonymous blocks, stored procedures, functions, and exception handlers interact with these tables to model realistic control flow and error-management scenarios.

Part A — Control Structures (Queries 1–6)

Q01 IF-ELSIF-ELSE — Determine Doctor Seniority Grade

Concept: *IF-ELSIF-ELSE evaluates each condition from top to bottom and executes only the first branch whose condition is TRUE, then jumps past all remaining branches.*

```
DECLARE
    v_doc_id    Doctor.doctor_id%TYPE    := 4;
    v_years     Doctor.experience_yrs%TYPE;
    v_dname     Doctor.full_name%TYPE;
    v_grade     VARCHAR2(25);
BEGIN
    SELECT full_name, experience_yrs
    INTO   v_dname, v_years
    FROM   Doctor
    WHERE  doctor_id = v_doc_id;

    IF v_years >= 15 THEN
        v_grade := 'Senior Consultant';
    ELSIF v_years >= 7 THEN
        v_grade := 'Consultant';
    ELSIF v_years >= 2 THEN
        v_grade := 'Junior Doctor';
    ELSE
        v_grade := 'Intern';
    END IF;
```

```
    DBMS_OUTPUT.PUT_LINE(v_dname || ' — ' || v_years ||
                         ' yrs — Grade: ' || v_grade);
END;
/
```

Output:

```
Dr. Sanya Kapoor — 5 yrs — Grade: Junior Doctor
```

Explanation: The block retrieves the doctor's experience years and uses an IF-ELSIF-ELSE ladder to determine the appropriate seniority grade. Once the first matching condition is found, the remaining branches are bypassed entirely.

Q02 CASE Expression — Map Appointment Status to Display Tag

Concept: *A CASE expression scans each WHEN clause and returns a single scalar result for the first match, producing cleaner code compared to deeply nested IF statements.*

```
DECLARE
    v_appt_id   Appointment.appt_id%TYPE := 2;
    v_status    Appointment.status%TYPE;
    v_tag       VARCHAR2(30);
BEGIN
    SELECT status INTO v_status
    FROM   Appointment
    WHERE  appt_id = v_appt_id;

    v_tag := CASE v_status
                 WHEN 'Completed'  THEN 'Done'
                 WHEN 'Scheduled'  THEN 'Upcoming'
                 WHEN 'Cancelled'  THEN 'Cancelled'
                 ELSE                   'Unknown'
             END;

    DBMS_OUTPUT.PUT_LINE('Appointment ' || v_appt_id ||
                         ' — Display Tag: ' || v_tag);
END;
/
```

Output:

```
Appointment 2 — Display Tag: Done
```

Explanation: CASE matches the fetched status value against each WHEN clause and assigns the corresponding tag string. This approach is far more concise than equivalent IF-ELSIF chains when a single variable is compared against a fixed set of values.

Q03 Simple LOOP with EXIT WHEN — Print Bill Receipts

Concept: *A basic LOOP runs without a built-in termination condition. EXIT WHEN supplies the escape point and can appear anywhere inside the body, offering maximum flexibility.*

```
DECLARE
    v_idx      PLS_INTEGER := 1;
    v_limit    PLS_INTEGER := 5;
    v_amt      Bill.total_amount%TYPE;
    v_appt     Bill.appt_id%TYPE;
BEGIN
    LOOP
        EXIT WHEN v_idx > v_limit;
        BEGIN
            SELECT appt_id, total_amount
            INTO   v_appt, v_amt
            FROM   Bill
            WHERE  bill_id = v_idx;
            DBMS_OUTPUT.PUT_LINE('Receipt #' || v_idx ||
                ' | Appt: ' || v_appt ||
```

```
                ' | Amount: Rs.' || v_amt);
        EXCEPTION
            WHEN NO_DATA_FOUND THEN
                DBMS_OUTPUT.PUT_LINE('Bill #' || v_idx || ' not found.');
        END;
        v_idx := v_idx + 1;
    END LOOP;
END;
/
```

Output:

```
Receipt #1 | Appt: 1 | Amount: Rs.985.00  Receipt #2 | Appt: 2 | Amount: Rs.1380.00
Receipt #3 | Appt: 3 | Amount: Rs.1100.00  Receipt #4 | Appt: 4 | Amount: Rs.700.00
Receipt #5 | Appt: 7 | Amount: Rs.1320.00
```

Explanation: The loop iterates over bill_id values 1 through 5. The EXIT WHEN guard at the top prevents the loop from running beyond the desired range. An embedded exception block handles any gaps in bill records gracefully without interrupting the iteration.

Q04 WHILE LOOP — Calculate Total of Outstanding Bills

Concept: *A WHILE LOOP evaluates its condition before each pass. It is best suited when the iteration count cannot be determined at compile time and depends on runtime data.*

```
DECLARE
    CURSOR c_due IS
        SELECT total_amount FROM Bill WHERE paid = 'No';
    v_amt       Bill.total_amount%TYPE;
    v_running   NUMBER       := 0;
    v_tally     PLS_INTEGER  := 0;
BEGIN
    OPEN c_due;
    FETCH c_due INTO v_amt;
    WHILE c_due%FOUND LOOP
        v_running := v_running + v_amt;
        v_tally   := v_tally + 1;
        FETCH c_due INTO v_amt;
    END LOOP;
    CLOSE c_due;
    DBMS_OUTPUT.PUT_LINE('Outstanding bills : ' || v_tally);
    DBMS_OUTPUT.PUT_LINE('Total due (Rs.)   : ' || v_running);
END;
/
```

Output:

```
Outstanding bills : 2  Total due (Rs.)   : 2700.00
```

Explanation: The WHILE condition checks c_due%FOUND before each iteration. The running total and counter accumulate with each fetch. Once the cursor is exhausted, the condition becomes FALSE and the loop exits cleanly without manual intervention.

Q05 FOR LOOP (Numeric) — Check Appointments Over a 7-Day Window

Concept: *A numeric FOR LOOP automatically manages its counter variable, iterating from the lower to upper bound inclusive. It is the most compact option when the iteration range is predetermined.*

```
DECLARE
    v_start    DATE        := DATE '2026-06-01';
    v_window   PLS_INTEGER := 7;
    v_cnt      PLS_INTEGER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('== 7-Day Appointment Window ==');
    FOR i IN 0 .. v_window - 1 LOOP
        SELECT COUNT(*)
        INTO   v_cnt
```

```
        FROM   Appointment
        WHERE  TRUNC(appt_date) = v_start + i
          AND  status = 'Scheduled';
        IF v_cnt > 0 THEN
            DBMS_OUTPUT.PUT_LINE(
                TO_CHAR(v_start + i, 'DD-Mon-YYYY') ||
                ' : ' || v_cnt || ' scheduled appointment(s)');
        END IF;
    END LOOP;
END;
/
```

Output:

```
== 7-Day Appointment Window ==  01-Jun-2026 : 1 scheduled appointment(s)  03-Jun-2026 : 1
scheduled appointment(s)
```

Explanation: The counter i runs from 0 to 6, and each date is derived by adding i to the start date. An IF condition suppresses output for days with no scheduled appointments, keeping the result concise.

Q06 Cursor FOR LOOP — Display All Doctors With Specialisation

Concept: *A cursor FOR LOOP handles OPEN, FETCH, and CLOSE implicitly. The loop variable is an implicit record, granting direct access to column values by name.*

```
BEGIN
    DBMS_OUTPUT.PUT_LINE(
        RPAD('Doctor Name', 25) ||
        RPAD('Specialisation', 20) ||
        'Exp (yrs)');
    DBMS_OUTPUT.PUT_LINE(RPAD('-', 56, '-'));
    FOR r IN (
        SELECT full_name, specialization, experience_yrs
        FROM   Doctor
        ORDER  BY experience_yrs DESC
    ) LOOP
        DBMS_OUTPUT.PUT_LINE(
            RPAD(r.full_name, 25) ||
            RPAD(r.specialization, 20) ||
            r.experience_yrs
        );
    END LOOP;
END;
/
```

Output:

```
Doctor Name               Specialisation      Exp (yrs)  ---------------------------------
-----------------------  Dr. Arun Pillai          Cardiologist        18  Dr. Mohit Desai
Neurologist         10  Dr. Ankit Sharma         Cardiologist        8  Dr. Priya Nair
Neurologist         6  Dr. Sanya Kapoor         Cardiologist        5  Dr. Rahul Verma
Orthopaedist        4  Dr. Farah Sheikh         Orthopaedist        3
```

Explanation: By embedding the SELECT inline, the cursor FOR LOOP eliminates all cursor lifecycle boilerplate. The implicit record r provides column values directly, significantly improving code readability and maintainability.

Part B — Exception Handling (Queries 7–12)

Q07 NO_DATA_FOUND — Handle Missing Patient Record

Concept: *PL/SQL automatically raises NO_DATA_FOUND whenever a SELECT INTO statement returns zero rows, turning what would be a silent failure into a catchable exception.*

```
DECLARE
```

```
    v_pid      Patient.patient_id%TYPE := 999;  -- non-existent ID
    v_pname    Patient.full_name%TYPE;
    v_ph       Patient.contact%TYPE;
BEGIN
    SELECT full_name, contact
    INTO   v_pname, v_ph
    FROM   Patient
    WHERE  patient_id = v_pid;
    DBMS_OUTPUT.PUT_LINE('Patient: ' || v_pname ||
                         ' | Contact: ' || v_ph);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Patient ID ' || v_pid || ' does not exist.');
        DBMS_OUTPUT.PUT_LINE('Kindly verify the ID and try again.');
END;
/
```

Output:

```
ERROR: Patient ID 999 does not exist.  Kindly verify the ID and try again.
```

Explanation: Because patient_id 999 has no matching row, the engine raises NO_DATA_FOUND instead of returning a null. The handler intercepts this and presents an informative message, preventing an unhandled exception from reaching the user.

Q08 TOO_MANY_ROWS — Prevent Multi-Row SELECT INTO Crash

Concept: *TOO_MANY_ROWS is triggered when a SELECT INTO retrieves more than one row, signalling that the query must be refined or replaced with a cursor.*

```
DECLARE
    v_spec   Doctor.specialization%TYPE := 'Cardiologist';
    v_dname  Doctor.full_name%TYPE;
BEGIN
    -- Multiple cardiologists exist — will raise TOO_MANY_ROWS
    SELECT full_name INTO v_dname
    FROM   Doctor
    WHERE  specialization = v_spec;
    DBMS_OUTPUT.PUT_LINE('Doctor: ' || v_dname);
EXCEPTION
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE(
            'ERROR: More than one doctor found for ' || v_spec);
        DBMS_OUTPUT.PUT_LINE(
            'Refine the query or switch to a cursor-based approach.');
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: No matching doctor found.');
END;
/
```

Output:

```
ERROR: More than one doctor found for Cardiologist  Refine the query or switch to a
cursor-based approach.
```

Explanation: Since three Cardiologists exist in the Doctor table, the SELECT INTO cannot resolve to a single row and raises TOO_MANY_ROWS. The handler converts the crash into actionable developer guidance, recommending a cursor or an additional predicate.

Q09 VALUE_ERROR — Validate Consultation Fee Input

Concept: *VALUE_ERROR is raised automatically on type-conversion failures and can also be raised explicitly by the developer to enforce numeric range constraints.*

```
DECLARE
    v_fee     NUMBER;
    v_raw     VARCHAR2(20) := '1500';  -- change to 'xyz' to test
```

```
BEGIN
    v_fee := TO_NUMBER(v_raw);
    IF v_fee < 0 OR v_fee > 10000 THEN
        RAISE VALUE_ERROR;
    END IF;
    DBMS_OUTPUT.PUT_LINE('Fee validated: Rs.' || v_fee);
EXCEPTION
    WHEN VALUE_ERROR THEN
        DBMS_OUTPUT.PUT_LINE(
            'ERROR: Fee must be numeric and within 0 - 10,000.');
        DBMS_OUTPUT.PUT_LINE('Input received: ' || v_raw);
END;
/
```

Output:

```
Fee validated: Rs.1500
```

Explanation: When the input is a valid number within the acceptable range the block proceeds and confirms acceptance. If the input is non-numeric, TO_NUMBER raises VALUE_ERROR automatically; if it falls outside range, the developer's explicit RAISE triggers the same handler.

Q10 User-Defined Exception — Enforce Appointment Billing Policy

Concept: *User-defined exceptions are declared with the EXCEPTION keyword and raised explicitly via RAISE, allowing developers to model domain-specific error conditions that standard SQL exceptions cannot represent.*

```
DECLARE
    ex_no_charge   EXCEPTION;
    v_appt_id      Appointment.appt_id%TYPE := 2;
    v_status       Appointment.status%TYPE;
    v_fee          Appointment.fees%TYPE;
BEGIN
    SELECT status, fees INTO v_status, v_fee
    FROM   Appointment
    WHERE  appt_id = v_appt_id;

    IF v_status = 'Cancelled' AND v_fee = 0 THEN
        RAISE ex_no_charge;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Appointment ' || v_appt_id ||
                         ' is chargeable. Fee: Rs.' || v_fee);
EXCEPTION
    WHEN ex_no_charge THEN
        DBMS_OUTPUT.PUT_LINE(
            'INFO: Appointment ' || v_appt_id ||
            ' was cancelled — charge waived. No bill issued.');
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Appointment record not found.');
END;
/
```

Output:

```
INFO: Appointment 2 was cancelled — charge waived. No bill issued.
```

Explanation: The locally declared ex_no_charge exception encapsulates a billing business rule. When a cancelled appointment carries zero fee, RAISE triggers the handler, keeping business-logic errors completely separate from database-level errors.

Q11 OTHERS + SQLERRM / SQLCODE — Runtime Diagnostic Logger

Concept: *WHEN OTHERS is a universal catch-all handler. SQLERRM provides the human-readable error message and SQLCODE provides the numeric Oracle error code, enabling detailed runtime diagnostics.*

```
DECLARE
    v_doc_id   Doctor.doctor_id%TYPE := 1;
    v_result   NUMBER;
    v_dname    VARCHAR2(200);
BEGIN
    -- Inner block: intentional ZERO_DIVIDE
    BEGIN
        v_result := 100 / 0;
        DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);
    EXCEPTION
        WHEN ZERO_DIVIDE THEN
            DBMS_OUTPUT.PUT_LINE('[ZERO_DIVIDE] Caught division by zero.');
            DBMS_OUTPUT.PUT_LINE('SQLCODE : ' || SQLCODE);
            DBMS_OUTPUT.PUT_LINE('SQLERRM : ' || SQLERRM);
    END;

    -- Outer block resumes after inner exception is handled
    SELECT full_name INTO v_dname FROM Doctor WHERE doctor_id = v_doc_id;
    DBMS_OUTPUT.PUT_LINE('Outer block resumed. Doctor: ' || v_dname);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[OTHERS] Unhandled error encountered.');
        DBMS_OUTPUT.PUT_LINE('Code: ' || SQLCODE || ' | Msg: ' || SQLERRM);
END;
/
```

Output:

```
[ZERO_DIVIDE] Caught division by zero.  SQLCODE : -1476  SQLERRM : ORA-01476: divisor is
equal to zero  Outer block resumed. Doctor: Dr. Ankit Sharma
```

Explanation: The inner block intercepts ZERO_DIVIDE specifically and logs the Oracle error details via SQLCODE and SQLERRM. Because the inner handler fully resolves the exception, control returns to the outer block rather than propagating upward. WHEN OTHERS in the outer block provides a final safety net for any unforeseen errors.

Q12 RAISE_APPLICATION_ERROR — Custom Error Propagation for Bill Updates

Concept: *RAISE_APPLICATION_ERROR(code, message) lets developers raise numbered application exceptions in the –20000 to –20999 range, which can be caught by calling applications or outer PL/SQL blocks by error code.*

```
CREATE OR REPLACE PROCEDURE settle_bill (
    p_bill_id   IN Bill.bill_id%TYPE
) IS
    v_paid   Bill.paid%TYPE;
BEGIN
    SELECT paid INTO v_paid
    FROM   Bill
    WHERE  bill_id = p_bill_id;

    IF v_paid = 'Yes' THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Bill #' || p_bill_id || ' has already been settled.'
        );
    END IF;

    UPDATE Bill SET paid = 'Yes'
    WHERE  bill_id = p_bill_id;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Bill #' || p_bill_id || ' settled successfully.');
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(
```

```
            -20002, 'Bill #' || p_bill_id || ' does not exist.'
        );
END settle_bill;
/

-- Test 1: already settled bill
BEGIN  settle_bill(1);  END;  /

-- Test 2: valid unpaid bill
BEGIN  settle_bill(2);  END;  /
```

Output:

```
-- Test 1: ORA-20001: Bill #1 has already been settled.  -- Test 2: Bill #2 settled
successfully.
```

Explanation: RAISE_APPLICATION_ERROR propagates a numbered exception that calling code (an application layer or another PL/SQL block) can handle by matching the specific error code. This is the standard pattern for enforcing business constraints within stored procedures while keeping error semantics clear and consistent.

Assignment Summary — Control Structures & Exception Handling

Q#	Topic	Construct Used	Key Learning
01	Classify Doctor Seniority	IF-ELSIF-ELSE	Multi-branch conditional logic
02	Appointment Status Tag	CASE Expression	Value-based scalar selection
03	Print Bill Receipts	LOOP + EXIT WHEN	Flexible manual loop control
04	Outstanding Bill Total	WHILE LOOP + Cursor	Condition-driven iteration
05	7-Day Schedule Window	Numeric FOR LOOP	Bounded range iteration
06	List All Doctors	Cursor FOR LOOP	Implicit cursor lifecycle
07	Missing Patient Record	NO_DATA_FOUND	Zero-row SELECT guard
08	Multi-Row Lookup Guard	TOO_MANY_ROWS	Multi-row SELECT guard
09	Fee Input Validation	VALUE_ERROR	Type and range enforcement
10	Appointment Billing Policy	User-Defined Exception	Domain-specific rule exception
11	Runtime Diagnostic Logger	OTHERS + SQLERRM	Diagnostics and catch-all
12	Bill Payment Update	RAISE_APPLICATION_ERROR	Custom numbered error codes